Personal AI Finance Advisor

Complete Project Documentation

AI-powered budgeting, spending analysis and saving guidance

Technology: Python · Flask · HTML · CSS · JavaScript · Gemini API

Author: Ayush Kumar

Personal AI Finance Advisor — Complete Project Documentation

1. Project Information

Project Name: Personal AI Finance Advisor

Project Type: AI-powered web application

Primary Purpose: Personal budgeting, spending analysis and saving guidance

Backend: Python + Flask

Frontend: HTML + CSS + JavaScript

AI Service: Google Gemini API

Data Format: JSON

Version Control: Git + GitHub

---

2. Abstract

Personal AI Finance Advisor is a web-based application that uses generative artificial intelligence to provide personalized financial guidance from user-provided income and expense information.

The user enters monthly income, expense categories, planned savings and a financial goal. The application sends the information to a Flask backend. The backend validates the input and constructs a structured prompt for the Gemini API. Gemini analyzes the information and returns structured JSON containing budget information, spending analysis and saving suggestions. The frontend then presents these results to the user.

The project demonstrates the practical integration of a generative AI model into a traditional web application.

---

3. Problem Statement

Many people record income and expenses but may find it difficult to understand spending patterns or create a realistic monthly budget.

The project addresses this problem by providing a simple interface where users can enter their financial information and receive AI-generated guidance based on the information they provide.

The goal is not to replace professional financial services. The goal is to provide an accessible educational tool for understanding personal spending and budgeting.

---

4. Objectives

The major objectives are:

1. Build a simple web-based financial assistant.

2. Accept income and expense information from users.

3. Validate financial input on the server.

4. Integrate a generative AI API.

5. Use prompt engineering to produce structured financial analysis.

6. Return AI results as JSON.

7. Display the results through a responsive frontend.

8. Protect sensitive configuration such as API keys.

9. Maintain the project using Git and GitHub.

10. Create documentation suitable for a portfolio or academic demonstration.

---

5. Scope

Included

• Monthly income input

• Expense-category input

• Savings input

• Financial-goal input

• AI-powered analysis

• Budget guidance

• Spending analysis

• Saving suggestions

• JSON-based API communication

• Basic input validation

• Error handling

• Responsive frontend

• Local development

• Optional public tunneling for demonstrations

Outside the Current Scope

• Direct bank-account access

• Automatic transaction synchronization

• Guaranteed investment recommendations

• Automatic financial transactions

• Professional financial advisory services

• Complete accounting software functionality

---

6. Key Features

6.1 Budget Generation

The application uses the submitted income and spending information to generate a structured budgeting response.

6.2 Spending Analysis

The system analyzes categories such as rent, food, transport, dining, entertainment and utilities.

The generated response can explain where the user's money is going and identify areas that may require attention.

6.3 Saving Guidance

The AI generates practical saving suggestions based on the submitted financial situation.

6.4 Goal-Based Guidance

The selected financial goal is included in the prompt so that the generated response can be tailored to that goal.

6.5 Structured Response

The backend requests JSON output from the AI service.

This provides a predictable structure for frontend rendering.

---

7. Technology Stack

Python

Used for backend application logic.

Flask

Flask provides the HTTP server and application routes.

Gemini API

Gemini provides the generative AI capability used to analyze financial information.

HTML5

Provides the structure of the web interface.

CSS3

Provides styling and responsive layout.

JavaScript

Handles user interactions, form submission and dynamic result rendering.

JSON

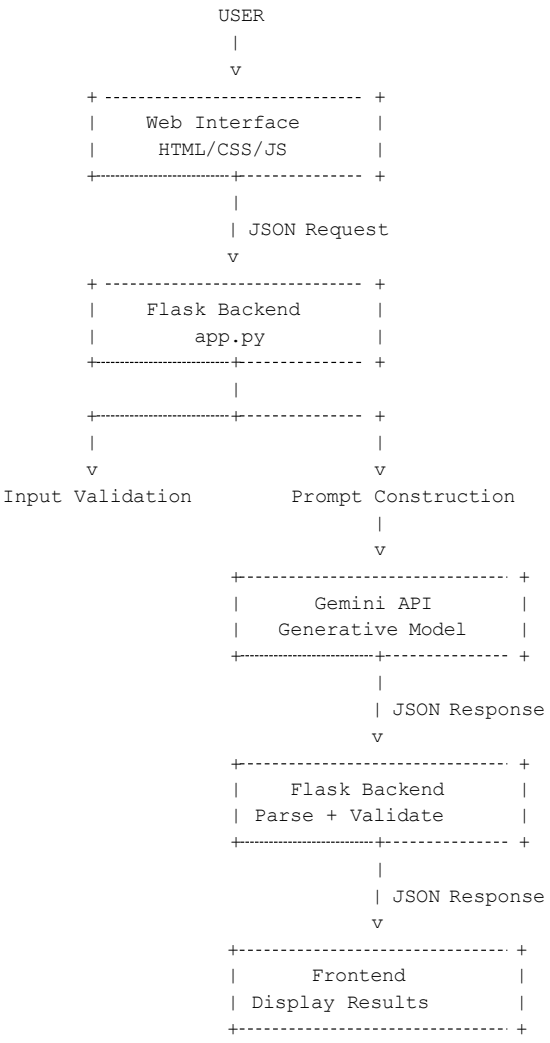
Used for communication between browser and backend and for structured AI output.

Git/GitHub

Used for version control, source-code management and project documentation.

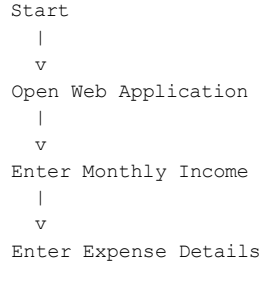
---

8. System Architecture



---

9. Application Workflow



```
    |
    v
  Select Financial Goal
    |
    v
  Submit Form
    |
    v
  JavaScript creates JSON
    |
    v
  POST /analyse
    |
    v
  Flask validates data
    |
    +----- Invalid ----> Return Error
    |
    v
  Build Financial Prompt
    |
    v
  Send Prompt to Gemini
    |
    v
  Receive AI Response
    |
    v
  Parse / Validate JSON
    |
    +----- Invalid ----> Return Error
    |
    v
  Return Structured Result
    |
    v
  Frontend Displays Analysis
     |
     v
   End
```

---

10. Project Directory

```
personal_finance_advisor/
■
■■■ app.py
■■■ requirements.txt
■■■ README.md
■■■ PROJECT_DOCUMENTATION.md
■■■ .env.example
■■■ .gitignore
■
■■■ templates/
■   ■■■ index.html
■
■■■ static/
    ■■■ css/
    ■   ■■■ style.css
    ■
    ■■■ js/
        ■■■ script.js
```

---

11. Backend Design

The main backend file is `app.py`.

The backend is responsible for:

1. Starting Flask.
2. Loading configuration.
3. Reading environment variables.
4. Rendering the home page.
5. Receiving JSON requests.
6. Validating user input.
7. Building the AI prompt.
8. Calling Gemini.
9. Parsing the returned JSON.
10. Returning the result to the frontend.
11. Handling errors and temporary API failures.

---

12. Flask Routes

/

Method

`GET`

Purpose

Loads the main application page.

Flow

```
Browser
   |
GET /
   |
Flask
   |
render_template()
   |
index.html
```

/analyse

Method

`POST`

Purpose

Processes the user's financial data and generates AI-powered financial guidance.

Flow

```
Frontend
   |
POST /analyse
   |
Read JSON
   |
Validate Input
   |
Build Prompt
   |
Gemini API
   |
Parse JSON
   |
Return Result
```

---

13. Input Validation

Input validation is performed before sending data to the AI service.

The application checks important conditions such as:

• Income is numeric.

• Income is greater than zero.

• Income is within the application's accepted upper limit.

• Request data exists.

• Financial values are usable.

This prevents obviously invalid data from being passed to the AI service.

---

14. Prompt Engineering

Prompt engineering is an important part of the application.

The backend constructs a prompt using the actual values entered by the user.

The prompt provides Gemini with:

• Income

• Expense categories

• Savings

• Financial goal

• Instructions for analysis

• Required output format

The prompt also instructs the model to return structured JSON.

This makes the AI output easier to consume programmatically.

---

15. Structured AI Response

The application expects the AI response to contain structured information such as:

```
{
  "budget": {},
  "analysis": [],
  "suggestions": [],
  "summary": {}
}
```

The exact content depends on the submitted financial data and AI response.

Structured output is useful because JavaScript can read specific properties and render them in the correct UI components.

---

16. Frontend Design

The frontend consists of three major layers.

HTML

Defines the page structure.

CSS

Defines:

• Layout

• Cards

• Typography

• Form appearance

• Buttons

• Responsive behavior

JavaScript

Handles:

• Form submission

• JSON request creation

• Fetch API calls

• Loading/error states

• Result rendering

---

17. Frontend-to-Backend Communication

The browser uses the Fetch API to communicate with Flask.

Conceptually:

```
Form
 ↓
JavaScript
 ↓
JSON
 ↓
POST /analyse
 ↓
Flask
 ↓
Gemini
 ↓
JSON response
 ↓
JavaScript
 ↓
UI
```

This approach avoids a full page reload when the user requests analysis.

---

18. Example Input

```
{
  "income": 50000,
  "expenses": {
    "rent": 15000,
    "food": 12000,
    "transport": 3000,
    "dining": 2500,
    "entertainment": 2500,
    "utilities": 3000,
    "savings": 10000
  },
  "goal": "emergency fund"
}
```

This represents an example monthly financial profile.

---

19. Example Financial Calculation

For the example above:

```
Income = ■50,000

Rent          = ■15,000
Food          = ■12,000
Transport     = ■3,000
Dining     =    ■2,500
Entertainment = ■2,500
Utilities     = ■3,000
Savings       = ■10,000
```

Total listed expenses including planned savings:

```
■15,000 + ■12,000 + ■3,000 + ■2,500
+ ■2,500 + ■3,000 + ■10,000
= ■48,000
```

Remaining amount from the stated figures:

```
■50,000 - ■48,000 = ■2,000
```

The AI can use these values as context when generating guidance.

---

20. Error Handling

The application should handle:

Missing API Key

If the Gemini API key is not configured, the application should return a configuration error rather than attempting an API call.

Invalid Input

Invalid financial input should result in a client-side or server-side validation response.

API Failure

Temporary Gemini service errors may be retried.

Rate Limiting

Temporary rate-limit responses can be handled with retry logic.

Invalid AI JSON

The backend validates/parses the AI response before returning it to the frontend.

---

21. Security

API Key

The API key should be stored in:

```
.env
```

and never directly inside source code.

Example:

```
GEMINI_API_KEY=your_api_key_here
```

Git Ignore

The `.gitignore` file should exclude:

```
.env
.venv/
venv/
__pycache__/
*.pyc
```

Important GitHub Rule

Never upload a real Gemini API key to GitHub.

If a real key has previously been committed, deleting it from the current working tree does not remove it from Git history. The key should be considered exposed and handled through the API provider's key-management controls.

---

22. Installation Procedure

Step 1 — Clone

```
git clone https://github.com/omwarade17/personal-ai-finance-advisor.git
cd personal-ai-finance-advisor
```

Step 2 — Create Virtual Environment

```
python -m venv .venv
```

Step 3 — Activate

```
.venv\Scripts\Activate.ps1
```

Step 4 — Install Packages

```
pip install -r requirements.txt
```

Step 5 — Create `.env`

```
GEMINI_API_KEY=your_api_key_here
```

Step 6 — Start Server

```
python app.py
```

Step 7 — Open Browser

```
http://127.0.0.1:5000
```

---

23. Testing Checklist

Functional Tests

- [ ] Application opens successfully.
- [ ] Income can be entered.
- [ ] Expense values can be entered.
- [ ] Goal can be selected.
- [ ] Form submission works.
- [ ] `/analyse` receives JSON.
- [ ] Gemini response is received.

• [ ] Results are rendered.

• [ ] Invalid input produces an error.

• [ ] Missing API key produces a useful error.

UI Tests

• [ ] Buttons work.

• [ ] Forms work.

• [ ] Results are readable.

• [ ] Layout works on desktop.

• [ ] Layout works on mobile.

• [ ] Error messages are visible.

Security Tests

• [ ] `.env` is ignored.

• [ ] API key is not present in source code.

• [ ] API key is not present in README.

• [ ] API key is not present in Git history.

---

24. Deployment

For development demonstrations, the Flask server can be exposed temporarily using ngrok.

Example:

```
python app.py
```

Then:

```
ngrok http 5000
```

This provides a temporary public URL that forwards requests to the local Flask application.

For production deployment, a proper hosting environment should be used instead of relying on a development Flask server or temporary tunnel.

---

25. Advantages

1. Simple user interface.

2. Easy-to-understand workflow.

3. AI-powered personalized responses.

4. Structured JSON output.

5. Lightweight Flask backend.

6. No database is required for the basic workflow.

7. Easy to run locally.

8. Useful educational application of generative AI.

9. Clear separation between frontend and backend.

10. API key can be protected through environment variables.

---

26. Limitations

1. AI-generated advice may not always be accurate.

2. User input may be incomplete.

3. No direct bank integration.

4. No automatic transaction tracking.

5. No persistent user account system in the basic implementation.

6. AI output can vary.

7. The application does not guarantee financial outcomes.

8. It is not professional financial, investment, tax or legal advice.

---

27. Future Scope

User Accounts

Add registration/login so users can maintain their own financial profiles.

Database

Store:

• Monthly income

• Expenses

• Goals

• Historical analyses

• Savings progress

Dashboard

Add charts for:

• Monthly expenses

• Category-wise spending

• Savings rate

• Budget vs actual spending

Reports

Allow users to export their financial analysis as PDF or CSV.

Goal Tracking

Allow users to set a target amount and track progress toward it.

Expense History

Provide monthly and yearly spending history.

Advanced AI

Improve prompt engineering and structured outputs while maintaining safe and transparent financial guidance.

Production Security

Add:

• HTTPS

• Authentication

• Rate limiting

• Secure secret management

• Logging

• Monitoring

• Automated tests

---

28. Educational Value

This project demonstrates several important software-development concepts:

Backend Development

Flask routes, request handling and JSON APIs.

Frontend Development

HTML, CSS and JavaScript-based interactive UI.

API Integration

Communication with an external generative AI service.

Prompt Engineering

Designing prompts that produce useful structured output.

Error Handling

Handling invalid input, API failures and malformed responses.

Environment Management

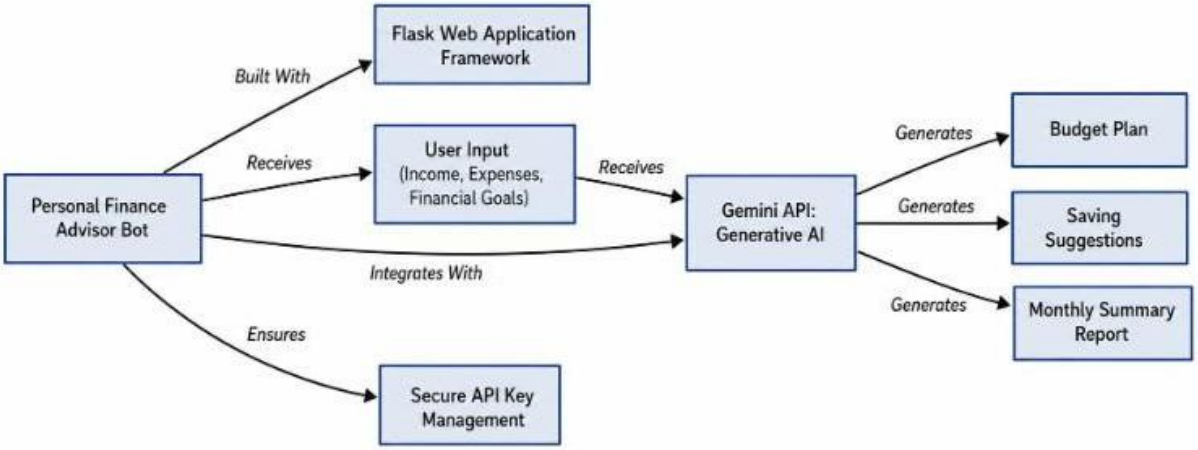
Using environment variables for secrets.

Version Control

Using Git and GitHub for project management.

---

29. Technical Architecture



30. Conclusion

Personal AI Finance Advisor demonstrates how generative AI can be integrated into a practical web application.

The project combines a Flask backend, responsive frontend and Gemini API to transform user-provided financial information into structured budgeting analysis and saving guidance.

The application provides a strong foundation for further development into a more complete personal-finance platform with user accounts, historical data, dashboards, goal tracking, reports and stronger production security.

The project also demonstrates practical skills in Python, Flask, JavaScript, REST-style communication, JSON processing, prompt engineering, API integration, error handling, Git and GitHub.